

UVM phases

UVM does not run your whole testbench in one initial block

Build, connect, run, check, report

- Build constructs the component tree
- Connect wires TLM ports and virtual interfaces, usually bottom-up after everything exists
- Run is where time advances
- When all objections drop, run ends and cleanup phases run
- Check is where scoreboards and self-checks decide pass versus fail
- Report prints summaries and error counts

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / UVM phase timeline

INTERACTIVE TOOL

UVM phase timeline

Step the usual UVM phase order: **build** → **connect** → **run** → **check** → **report** (with a few neighbors for context). Starter: cursor on run while an objection holds time open.

Starter example: timeline on the core path build → connect → run → check → report, with an objection raised so run stays open.

Load starter example

Challenges 0 / 22 cleared

Quiz: order: The core teaching order is...

☐ build → connect → run → check → report

☐ report → run → build → connect → check

☐ check → build → connect → run → report

☐ run → build → report only

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

build → connect

Topology first: create components, then wire them.

run

Where time advances; objections keep the phase alive.

check → report

After run: decide pass/fail, then summarize.

Direction

Build often top-down; many later phases bottom-up.

UVM phases lab starter

learn_uvm2017 — uvm phases

Real UVM literacy

```
wsl - module04-uvm-phases/examples/navigation

# Track A - real Unix
# real shell session (Track A - UVM phase literacy)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module04-uvm-phases

$ ls examples/phase-timeline/
phases.txt

$ cat examples/phase-timeline/phases.txt
UVM phase timeline sketch (literacy - core spine)


---



---



CORE PATH (memorize first):
  build   → construct components (factory create), top-down
  connect → wire TLM / virtual interfaces, bottom-up
  run     → time advances; sequences + drivers; objections hold run open
  check   → scoreboard / self-check pass vs fail
  report  → summaries, UVM error counts

Neighbors (awareness):
  end_of_elaboration - last build-time topology tweak
  start_of_simulation - just before time moves
  extract             - gather results after run
  final               - last cleanup

Run phase notes:
- Test raises objection before starting sequences
- Drop objection when stimulus/checking is done
- No objections → run may end immediately

Common mistakes:
- Driving DUT in build (belongs in run via driver)
- Forgetting connect after build (ports unwired)
- Ending run while sequences still active (objection dropped too early)

$ grep -n "phase" ../../learn_uvm2017_sv_verilator/module6/examples/architecture/architecture.sv | head -8
10: *   5. Understand UVM phases execution
28: *   - build_phase: Create components
29: *   - connect_phase: Connect components (driver-sequencer)
30: *   - run_phase: Execute test (sequences, driving, monitoring)
31: *   - report_phase: Generate reports
174: *   * Run phase: Drive transactions
188:   task run_phase(uvm_phase phase);
296:   //   - Created in constructor (before build_phase)
```

Real shell — phase timeline sketch

Pitfalls to watch

- Do not drive the DUT in build, build is for construction and configuration, not time
- Do not forget objections in run
- Do not assume connect and build walk the same direction
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter, walk the core five phases, and finish a few challenges
- On real UVM, recite build through report from memory and note where objections belong
- When you are ready, take the short quiz, then continue to the factory in the next module